

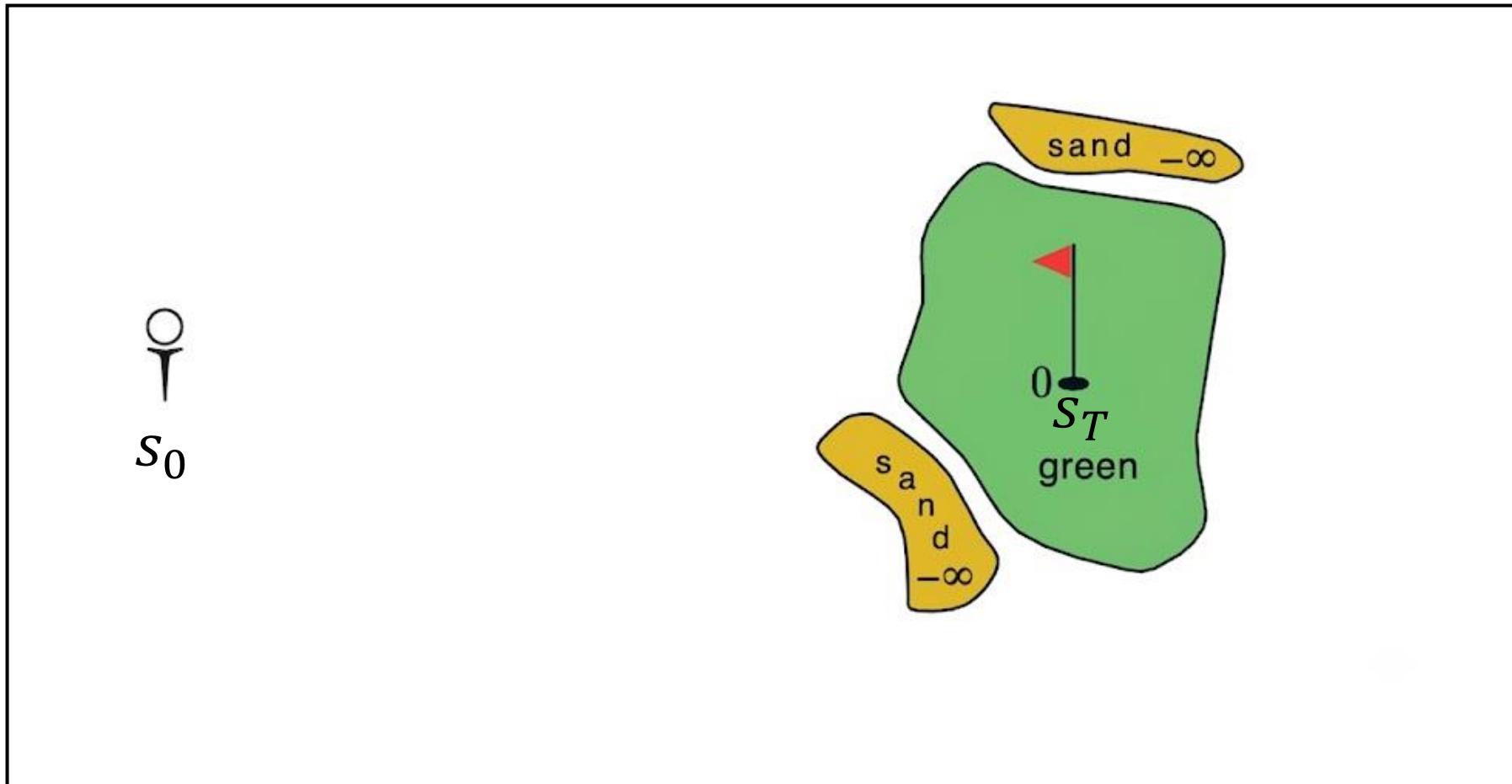
Reinforcement Learning: Planning by Dynamic Programming (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Example: Playing Golf

$$s \in S$$



$$a \in A$$



Driver

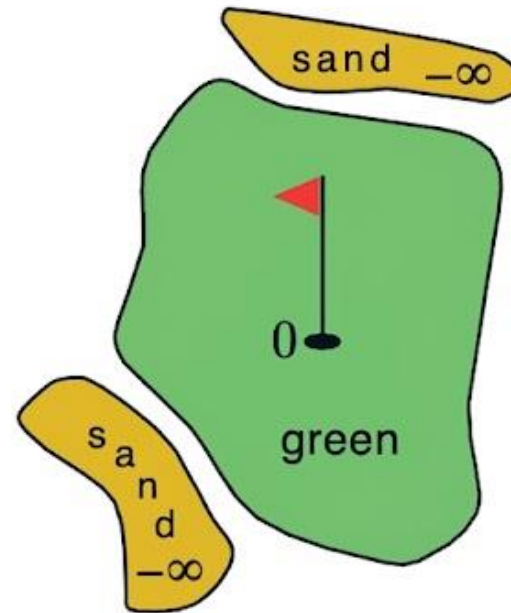


Putt

$$r(s, a) = -1$$

Playing Golf: V^π

$$s \in S$$



$$V^\pi(s_T) = ?$$

If $S^1 \subset S$ is the region of the court where I can reach the hole within one putt, what is $V^\pi(s), s \in S^1$?

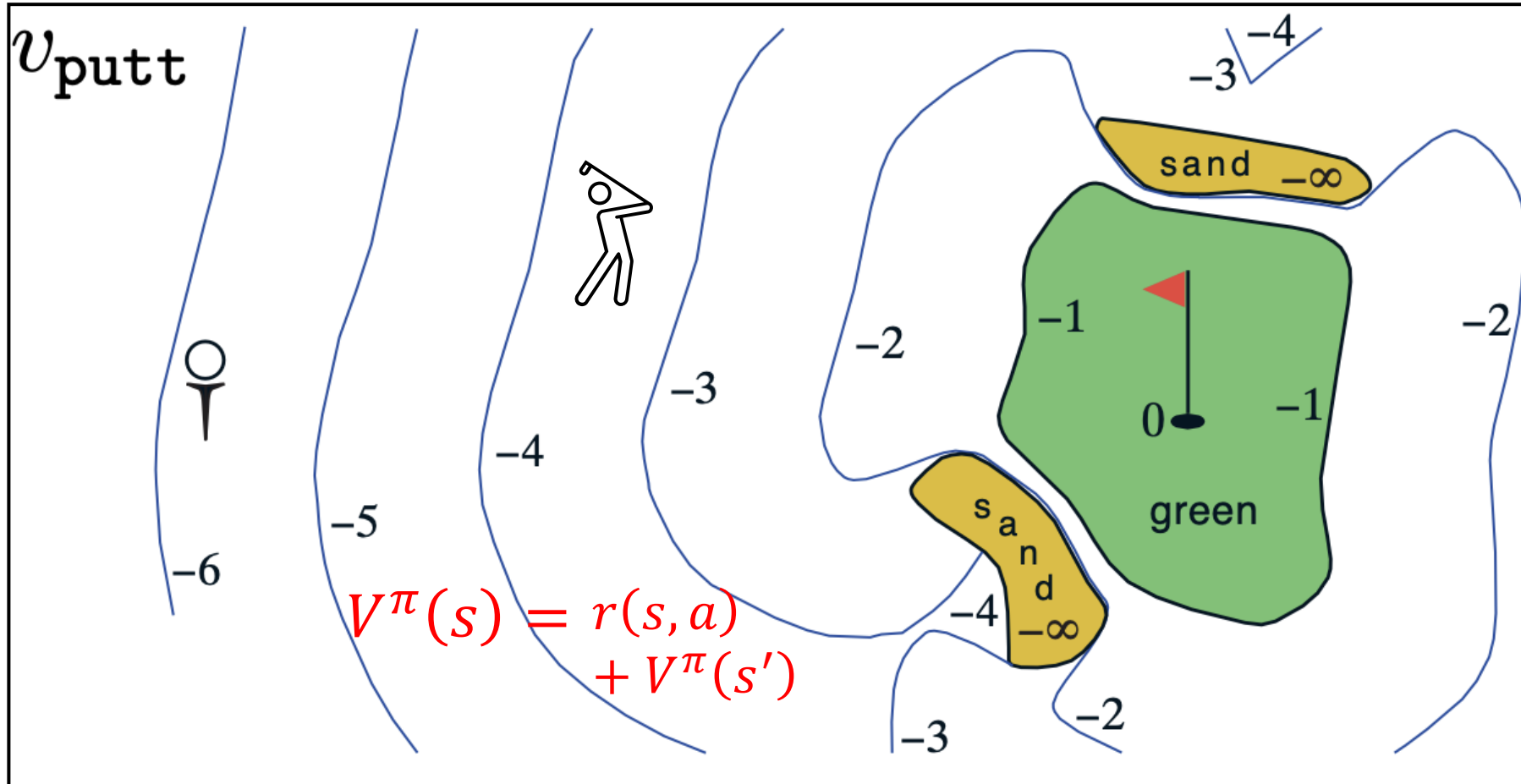
$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

The Bellman Expectation Equation

$$s \in S$$



$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

General Case: Bellman Expectation Equation

- For a *discrete state and action set*:

$$V^\pi(s) = r_{t+1} + \gamma \sum_{s' \in S} P_{ss'}^\pi V^\pi(s')$$

- ***Intuition***. Value of current state is current reward plus the discounted value of the next state I end up in, *averaged* by the probability of reaching that state.

General Case: Bellman Expectation Equation

- For a *continuous state and action set*:

$$V^\pi(s) = r_{t+1} + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^\pi(s_{t+1}) | s, \pi(s)]$$

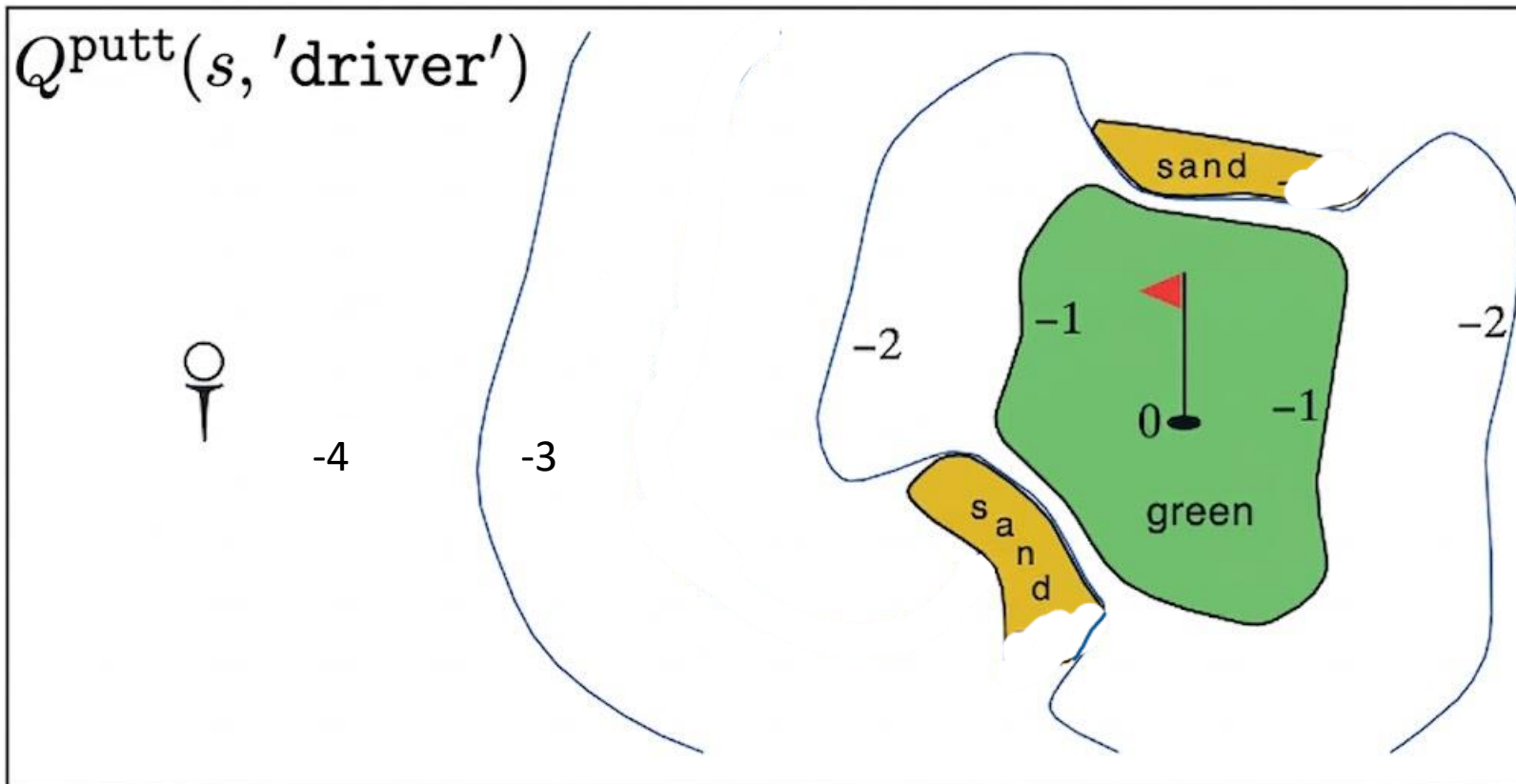
- ***Intuition.*** Value of current state is current reward plus the discounted *expected* value of the state I end up in.

Keeping your options open: Q^π

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

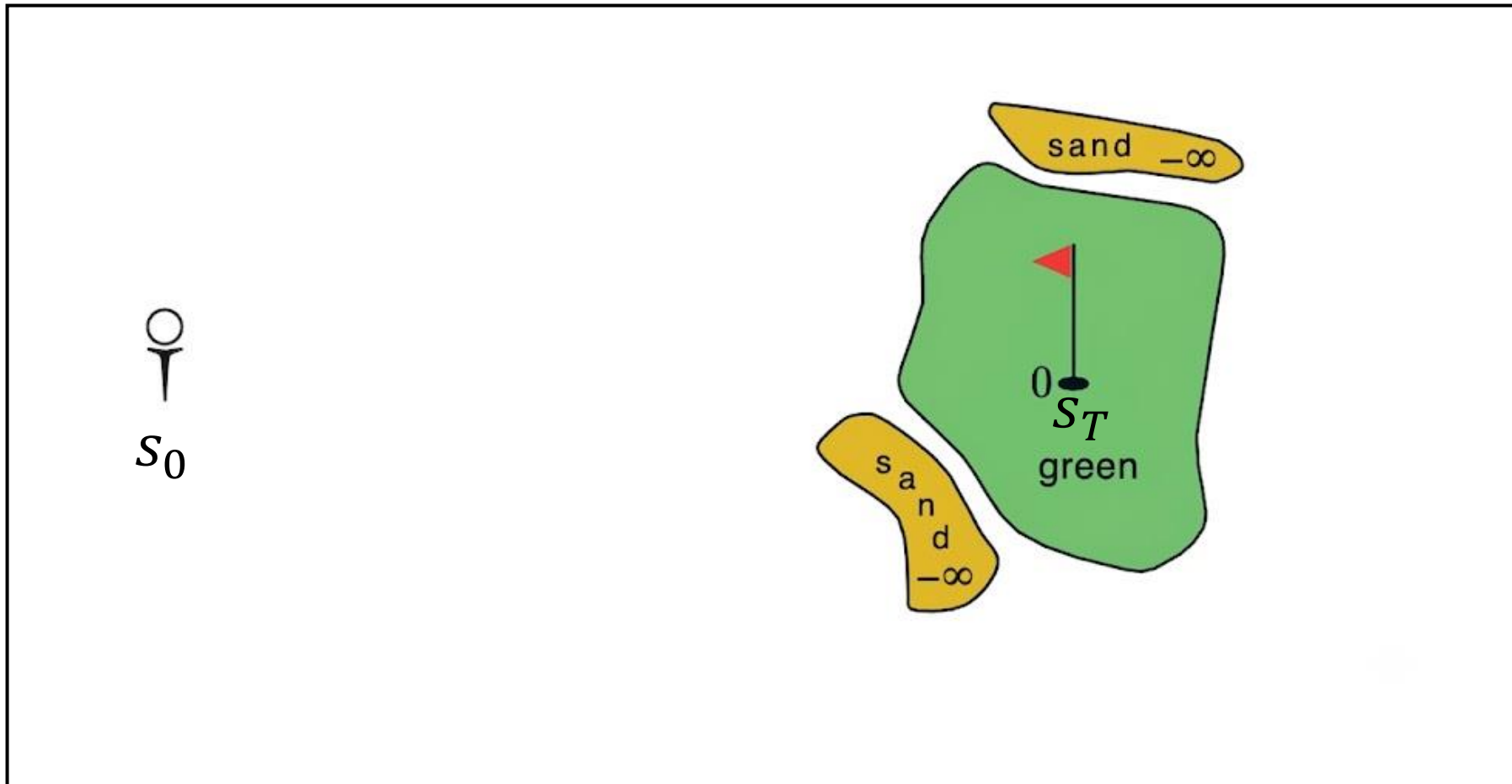


- $Q^\pi(s, a)$:

First you take action a , then always use the putt

Playing Golf with an Expert: $Q^*(s, a)$

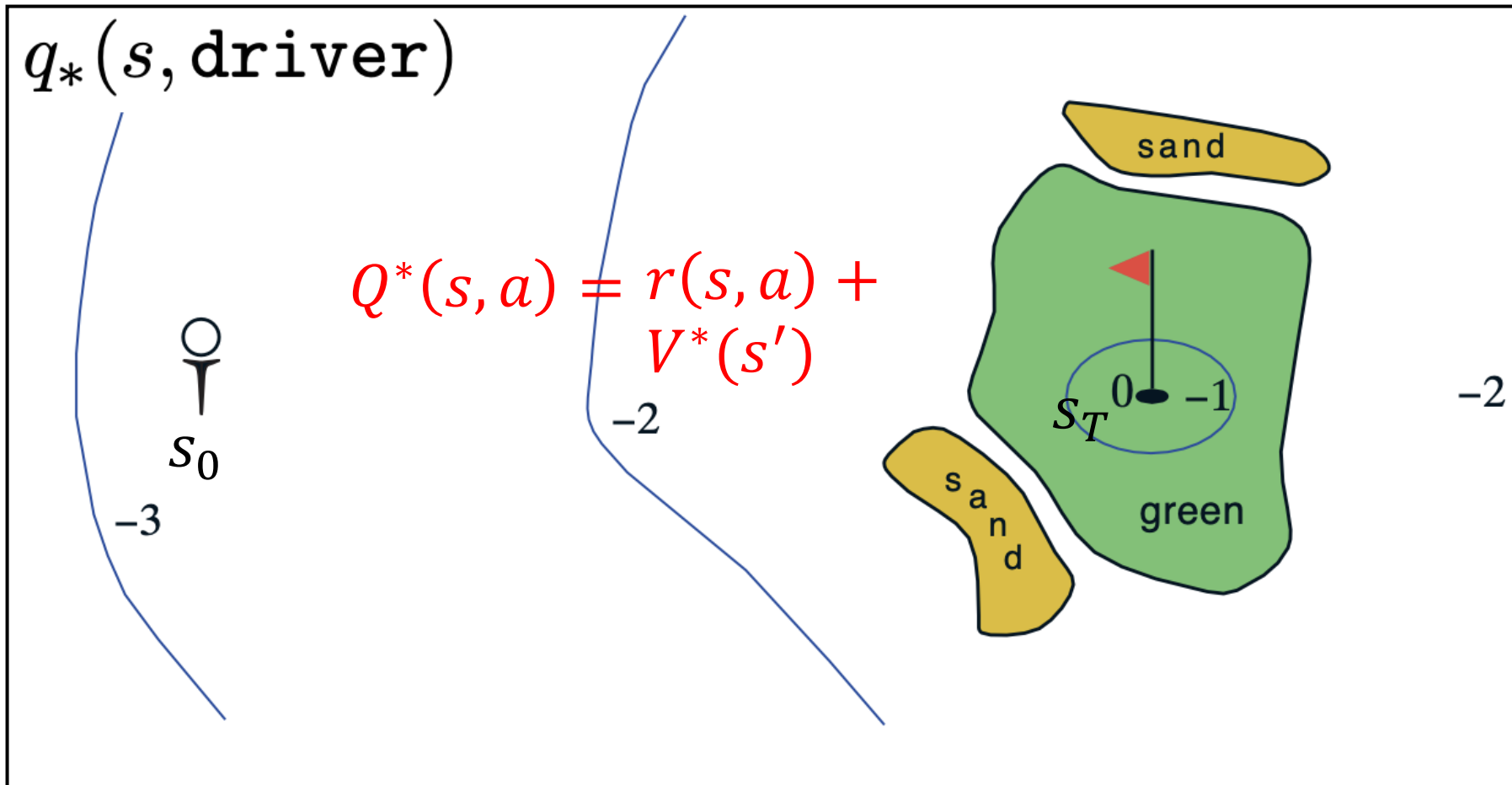
$$s \in \mathcal{S}$$



$Q^*(s, a)$:

- You can take a single action, then the expert takes over.

Visualizing the Bellman Optimality Equation



Only action for you



Driver

Playing Golf with an Expert

- What is your expected return?

$$Q^*(s, a = \text{driver}) = r(s, a = \text{driver}) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [\max_{\pi} V^{\pi}(s_{t+1}) | s, a]$$

Bellman Equation: Expectation vs Optimality

- *Expectation equation: Follows a policy π .*

$$V^\pi(s) = r(s, \pi(s)) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^\pi(s_{t+1}) | s, \pi(s)]$$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^\pi(s_{t+1}) | s, a]$$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [Q^\pi(s_{t+1}, \pi(s_{t+1})) | s, a]$$

- *Optimality Equation: Follows the optimal policy π^* .*

$$V^*(s) = r(s, \pi^*(s)) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^*(s_{t+1}) | s, \pi^*(s)]$$

$$= r(s, \pi^*(s)) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [\max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) | s, \pi^*(s)]$$

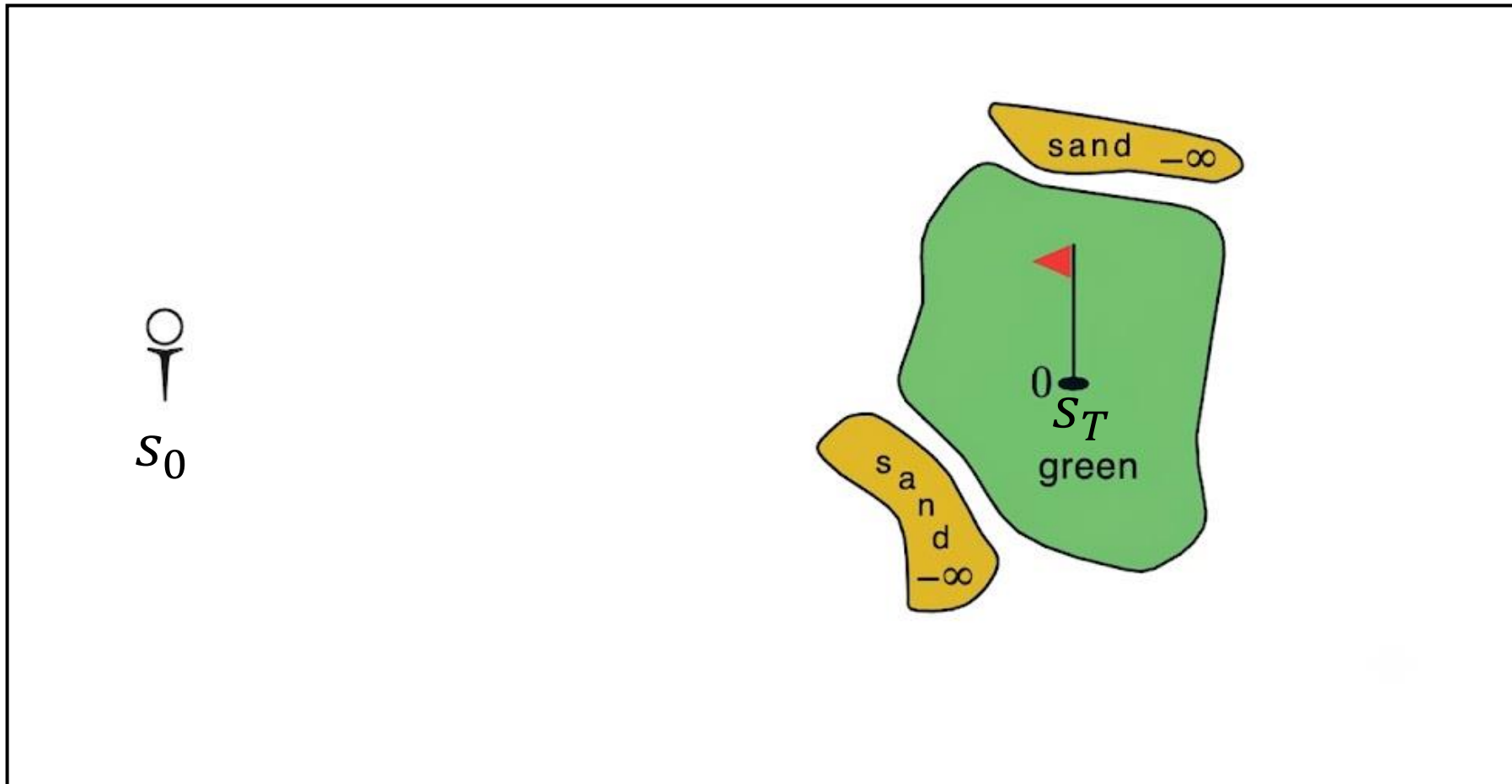
$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [\max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) | s, a]$$

What We Will Cover in the Next Classes

- Derive a Value Function and Policy of a *known* MDP
 - Planning by Dynamic Programming
- Evaluate the value function of an *unknown* MDP
 - Model-free prediction
- Optimize the value function/policy of an *unknown* MDP
 - Model-free control

Dynamic Programming: Playing Golf

$$s \in S$$



$$a \in A$$



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Dynamic Programming: Assumptions

- In dynamic programming, you assume to have perfect knowledge of:
 - The environment (in our case, the map of the golf course)
 - The transition dynamics (where a ball lands after taking an action, in expectation)
 - The reward structure (i.e., you know $r(s, a) \forall s, a$).
- In short, you have full knowledge of the MDP describing the agent-environment interaction.



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Dynamic Programming: Playing Golf

- Finding $V^*(s)$. Algorithm intuition:
 - Start from the states *you know* you can reach the hole with a single action (either putt or driver). Let's call this set $S^1 \subset S$.
What is $V^*(s), \forall s \in S^1$?
 - Now go to the states you know you can reach a state in S^1 with a single action. Let's call this set $S^2 \subset S$.
What is $V^*(s), \forall s \in S^2$?
 - Continue up to when you have covered *all states* in S .
Return $V^*(s), \forall s \in S$.



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Dynamic Programming: Playing Golf

- Finding $\pi^*(s)$. Algorithm intuition:
 - Step 1: First find $V^*(s)$ using the algorithm presented above.
 - Step 2: $\forall s \in S$, find $\pi^*(s)$ as:

$$\pi^*(s) = \arg \max_a r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^*(s_{t+1})|s, a]$$

- **Quiz:** Is the following true?

$$\begin{aligned} \pi^*(s) \\ = \arg \max_a r(s, a) + \gamma \arg \max_a \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^*(s_{t+1})|s, a] \end{aligned}$$



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Dynamic Programming: Playing Golf

- Algorithm intuition:
 - Step 1: First find $V^*(s)$ using recursive search.
 - Step 2: $\forall s \in S$, find $\pi^*(s)$ as:
$$\pi^*(s) = \arg \max_a r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^*(s_{t+1}) | s, a]$$
- Note that we did not have to hit a single ball to arrive to this conclusion!
 - We only used our initial knowledge to produce an optimal plan of actions.
- In general, dynamic programming requires **NO** environment interactions (zero samples!).



Driver



Putt

$$r(s, a) = -1$$

What is Dynamic Programming?

- **Dynamic**: sequential or temporal component to the problem
- **Programming**: optimizing a “program”, i.e., a policy.
- Dynamic Programming is a method for solving complex problems by breaking them down into subproblems.
 - We first solve the subproblems, and then combine their solutions.

Requirements for Dynamic Programming

- Dynamic Programming is a very general solution method for problems with two properties:
 - **Optimal Substructure:**
 - The optimal solution can be decomposed into subproblems.
 - Any sub-problem of an optimal solution is itself optimal (*principle of optimality*)
 - **Overlapping Subproblems:**
 - Subproblems *recur* many times. Therefore, solutions can be cached and reused.
- MDPs satisfy both properties
 - Bellman equation gives recursive decomposition.
 - Value function stores and reuses solutions.

Planning by Dynamic Programming

- Dynamic Programming assumes full knowledge of the MDP
- It is used for *planning* in an MDP.
- Examples:
 - **Prediction**. Given an MDP and a policy π , *output* a value function V^π .
 - **Control**. Given an MDP, *output* the optimal value function V^* and policy π^*

Applications of Dynamic Programming

- Despite its apparent strong assumptions, Dynamic programming is used in many applications:
 - Scheduling algorithms (your class time!)
 - String algorithms (e.g., sequence alignment)
 - Graph algorithms (e.g., shortest path algorithms)
 - Graphical models (e.g., Viterbi algorithm)
 - Bioinformatics (e.g., lattice models)

Prediction: Find V^π given π

- Isn't this trivial for known MDPs with discrete states and actions?

$$\begin{aligned}V^\pi &= \mathbf{R} + \gamma \mathbf{P} V^\pi \\V^\pi &= (\mathbf{I} - \gamma \mathbf{P})^{-1} \mathbf{R}\end{aligned}$$

- Where:
 - $V^\pi = [V^\pi(s_0), V^\pi(s_1), \dots, V^\pi(s_n)]$ is the vector containing the value for all possible states.
 - $\mathbf{P}$ is the transition probability matrix
 - $\mathbf{R} = [R(s_0, \pi(s_0)), R(s_1, \pi(s_1)), \dots, R(s_n, \pi(s_n))]$ is the vector containing the immediate reward for each state.
- Why do we need this Dynamic Programming stuff?

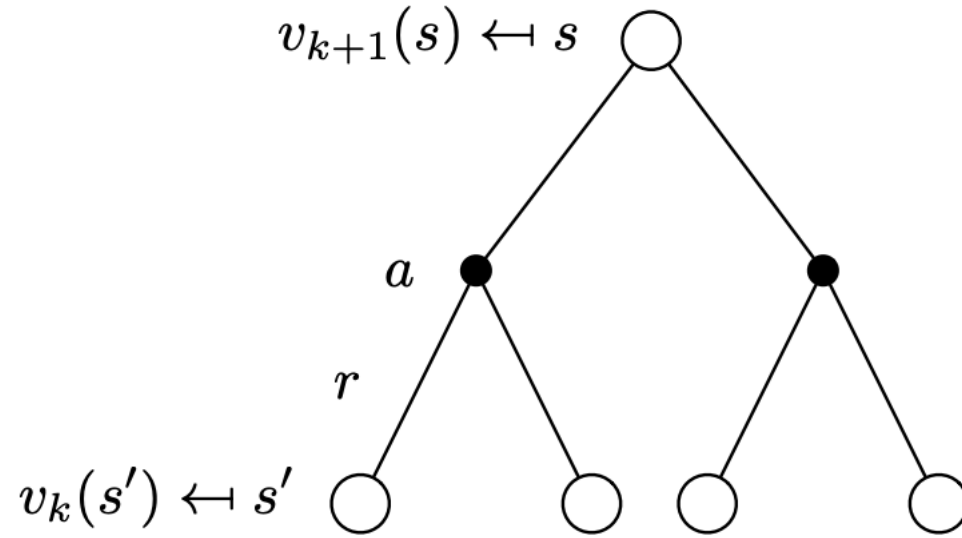
Iterative Policy Evaluation: V^π w/o Matrix Inversion

- Problem: Given a policy π , find its value function V_π .
- Algorithm (*synchronous* backups):
 - Initialize $V_0 = 0$
 - At each iteration $k + 1$
 - $\forall s \in S$, update $V_{k+1}(s)$ from $V_k(s')$, where s' is a successor state of s


$$V_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s') \right)$$

- **Idea:** Iterative application of Bellman expectation backup.
$$V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V^\pi$$
- The intermediate V_k are not necessarily value function of any policy.
- We can prove asymptotic convergence to V^π

Iterative Policy Evaluation



$$V_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s') \right)$$

$$\mathbf{V}_{k+1} = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{V}_k \leftarrow \text{No inversion!}$$

Iterative Policy Evaluation: When to Stop?

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

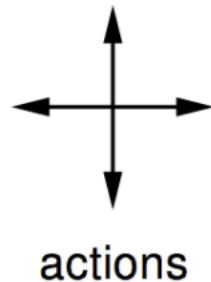
$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Example: Evaluating a Random Policy

- Characteristics:
 - Undiscounted MDP, $\gamma = 1$
 - Terminal states in gray
 - Action leading out of the grid leave the state unchanged
 - $r(s, a) = -1$
 - Agent follows a random policy: $\pi(s, a) = 0.25, \forall s, \forall a$



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$r = -1$
on all transitions

Example: Evaluating a Random Policy

4	O	O	O	O
3	O	O	O	O
2	O	O	O	O
1	O	O	O	O
	A	B	C	D

$k = 0$

4	O	-1	-1	-1
3	-1	-1	-1	-1
2	-1	-1	-1	-1
1	-1	-1	-1	O
	A	B	C	D

$k = 1$

4	O	-1.7	-2	-2
3	-1.7	-2	-2	-2
2	-2	-2	-2	-1.7
1	-2	-2	-1.7	O
	A	B	C	D

$k = 2$

4	O	-2.4	-2.9	-3
3	-2.4	-2.9	-3	-2.9
2	-2.9	-3	-2.9	-2.4
1	-3	-2.9	-2.4	O
	A	B	C	D

$k = 3$

4	O	-6.1	-8.4	-9
3	-6.1	-7.7	-3	-8.4
2	-8.4	-3	-7.7	-6.1
1	-9	-8.4	-6.1	O
	A	B	C	D

$k = 10$

4	O	-14	-20	-22
3	-14	-18	-20	-20
2	-20	-20	-18	-14
1	-22	-20	-14	O
	A	B	C	D

$k = \infty$

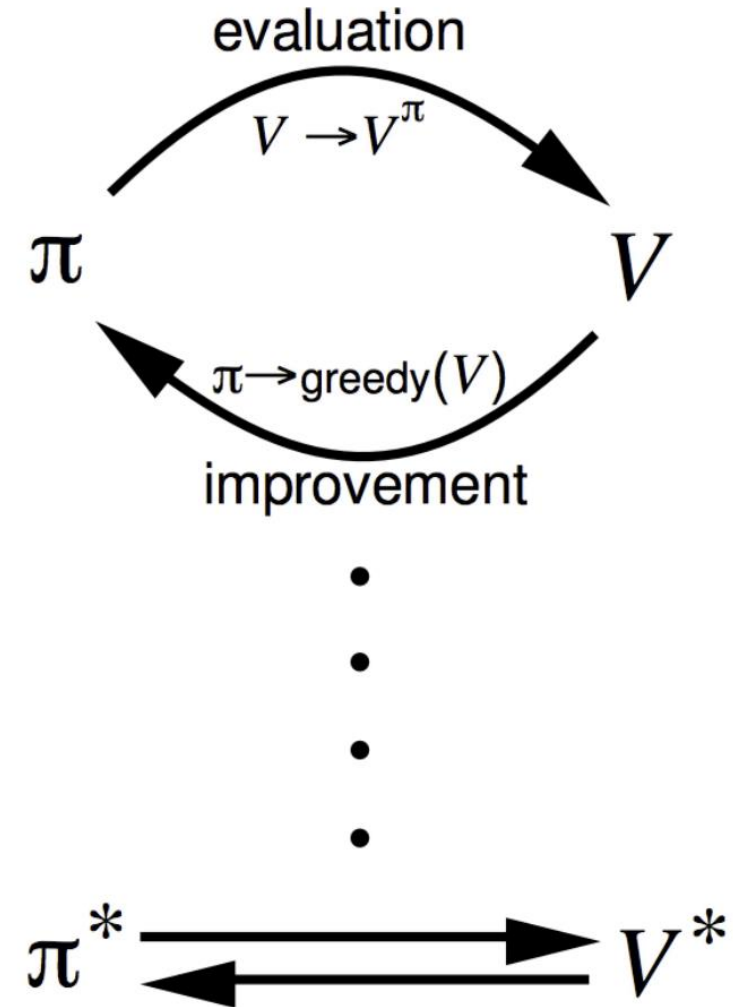
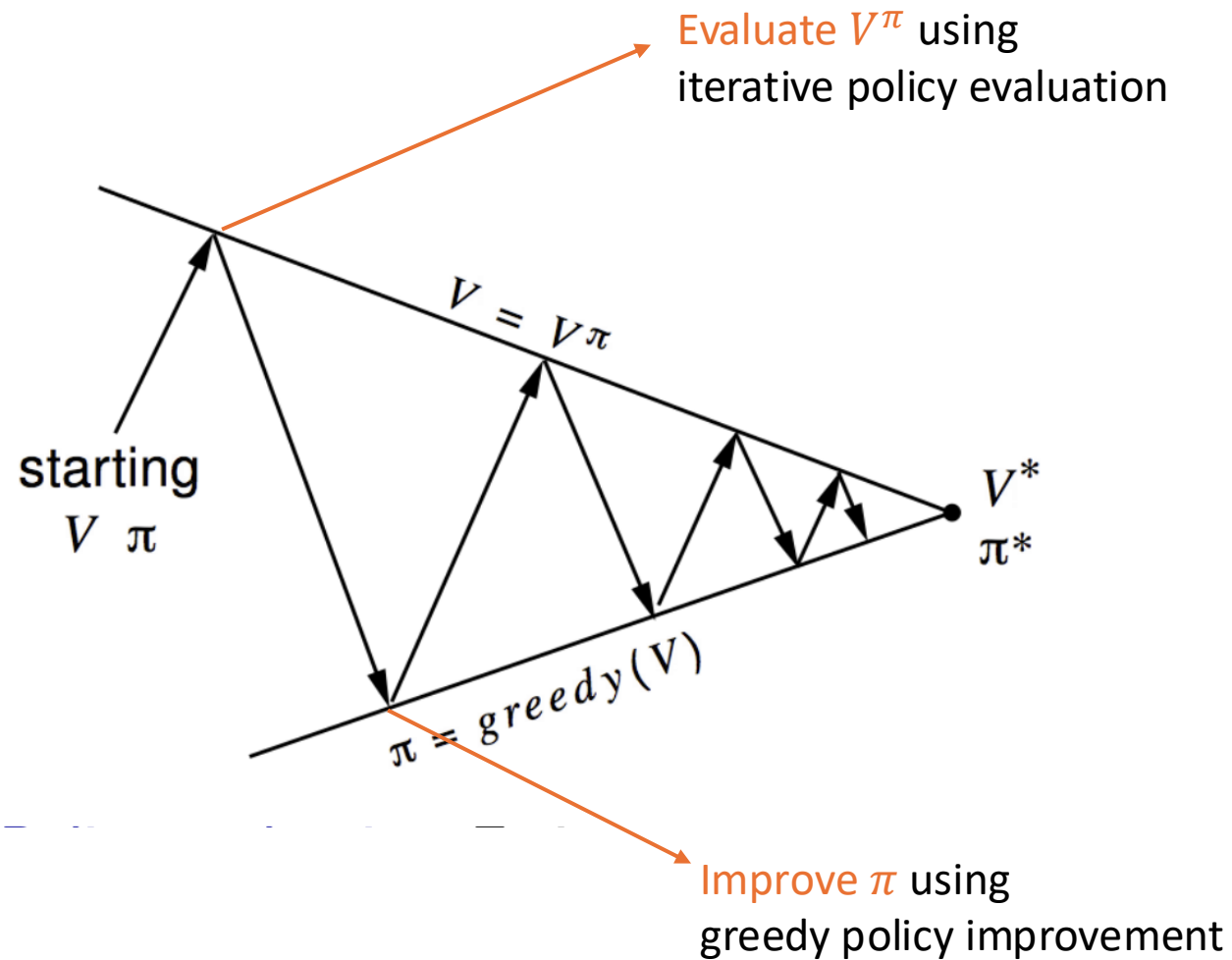
How to Improve a Policy: Policy Iteration

- Start with a random policy π
- **Evaluate** the policy π , using the algorithm above.
- **Improve** the policy by acting *greedily* with respect to V_π

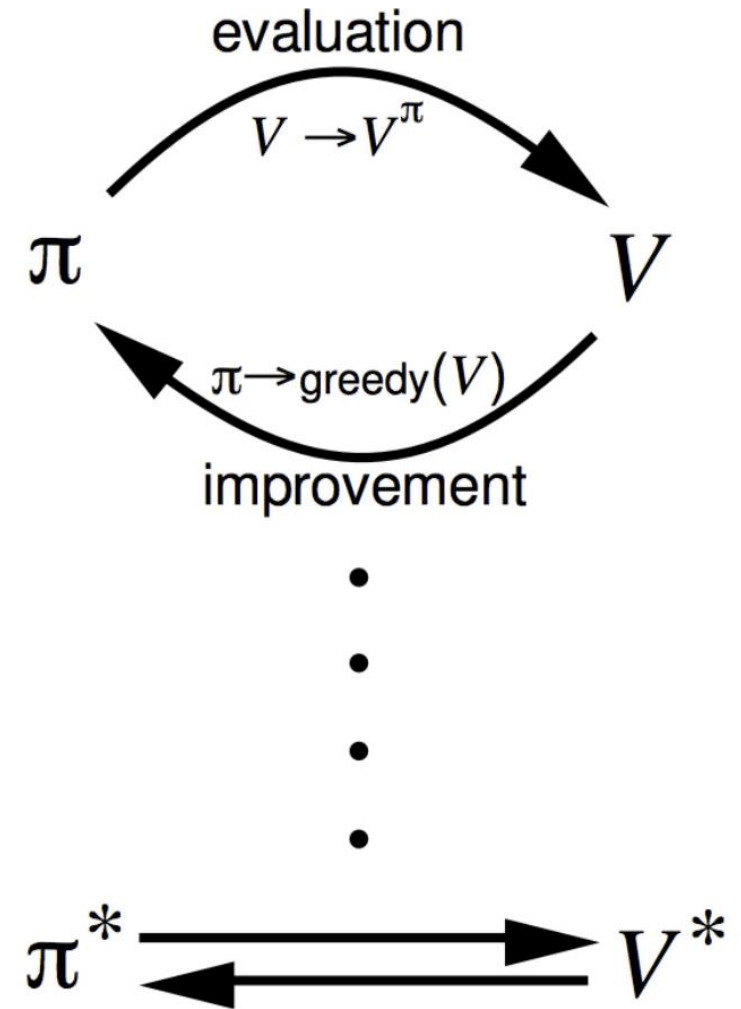
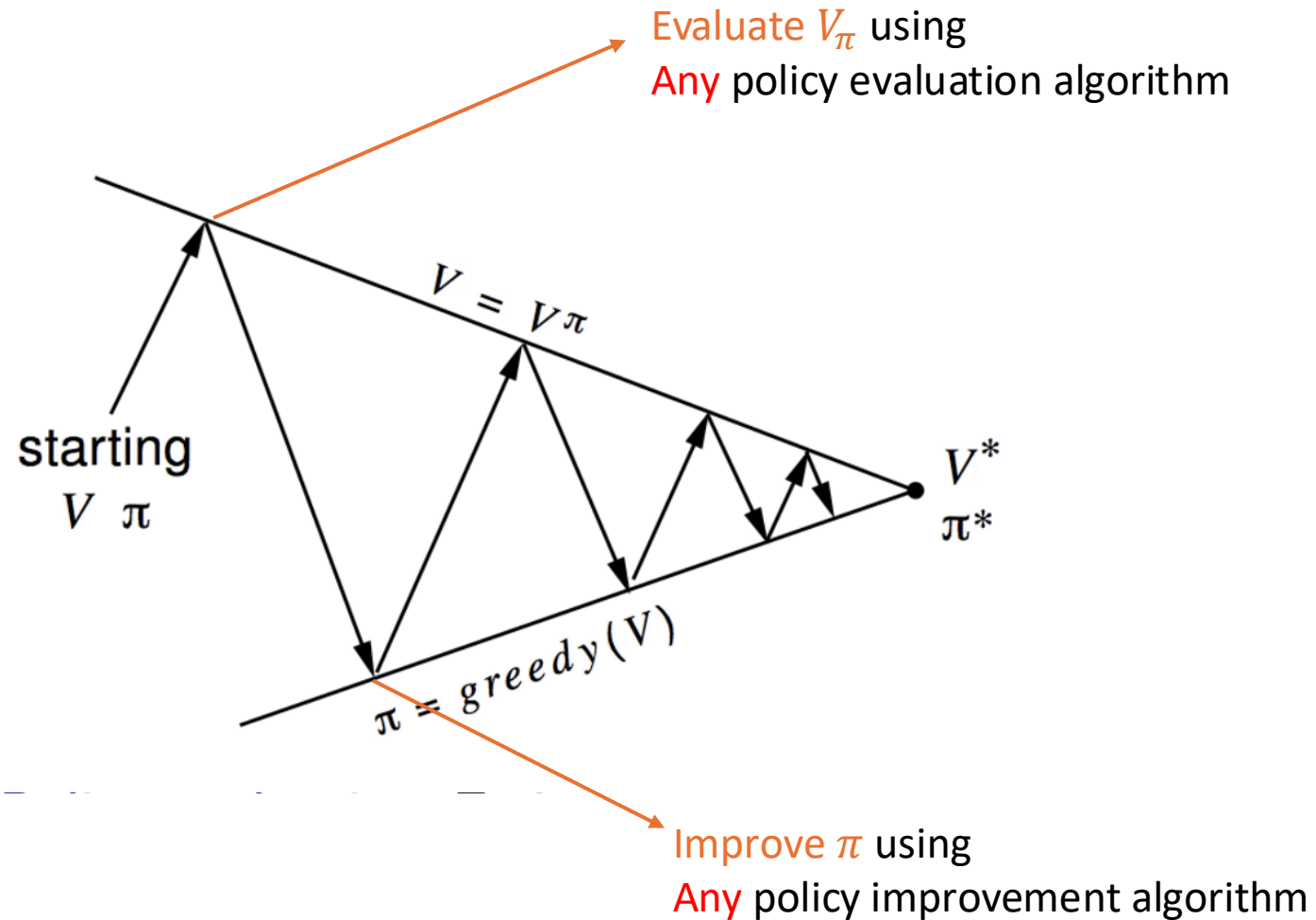

$$\pi'(s, a) = \begin{cases} 1 & \text{if } a = \operatorname{argmax} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_\pi(s') \\ 0 & \text{otherwise} \end{cases}$$

- In the gridworld example above, a single iteration gives you the optimal policy. But in general, you need more iterations
- However, policy iteration always converges to the optimal policy.

Policy Iteration



Generalized Policy Iteration



Can we do better? *Modified* Policy Iteration

- Do we need to compute the exact V^π after each policy update?
- To *save* computation/time, we could:
 - Introduce a large stopping condition, as for example ϵ -convergence to the value function
 - or stop after a fixed number k of iterative policy evaluations

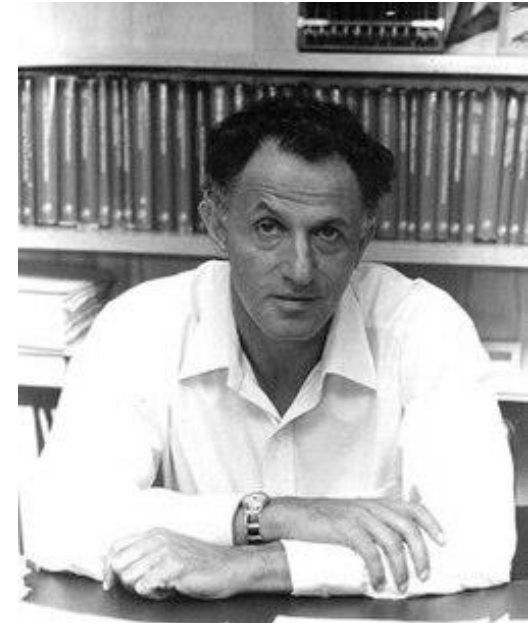
Modified Iterative Policy Evaluation



Great Implementation! But feels hacky...

- The idea of modified policy iteration is great for saving compute, but feels hacky.
- Can we get an algorithm that has similar or better compute requirements, but is still guaranteed to converge to V^*, π^* ?
- Yes! We can do as little as one evaluation step ($k = 1$).
- This algorithm is called **Value Iteration**.

Principle of Optimality (Bellman, '57)



- An optimal policy has the property that:
 - whatever the initial state and initial decision are
 - the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.
- This is called *Optimal Substructure*.
- Can you give an example of a problem w/o optimal substructure?

Principle of Optimality (Bellman, '57)

- Any optimal policy can be *subdivided* into two components:
 - An optimal first action a_*
 - Followed by an optimal policy from successor state s'
- **Theorem** (*Principle of Optimality*): A policy $\pi(a|s)$ achieves the optimal value from state s , $V^\pi(s) = V^*(s)$, *if and only if*
 - For any state s' reachable from s , π achieved the optimal value from s' , $V^\pi(s') = V^*(s')$

Principle of Optimality (Bellman, '57)

An important consequence of the principle of optimality:

- If we know the solution to subproblems $V_*(s')$
- Then the solution to $V(s)$ can be found *with a one-step lookahead*:

$$V^*(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V^*(s')$$

- Start with final states and work *backwards*. Remember the golf example?
- The idea of **Value Iteration** is applying the same formula, but with V_k :

$$V_{k+1}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s')$$

Value Iteration: Finding V^*

- Idea: Iterative application of Bellman optimality backup

$$V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V^*$$

- Use synchronous backups



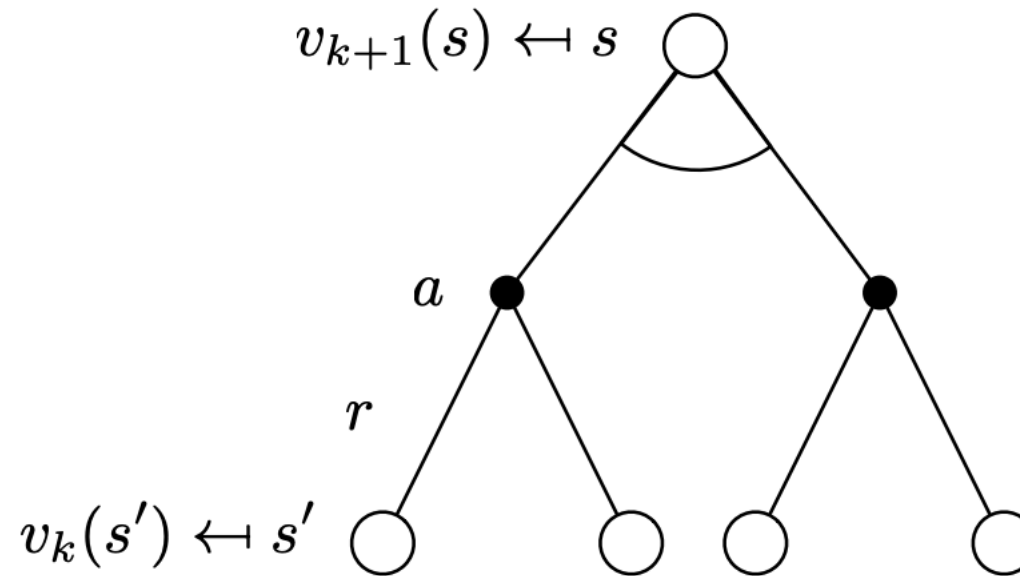
- At each iteration $k + 1$

- $\forall s \in S$, update $V_{k+1}(s)$ from $V_k(s')$:

$$V_{k+1}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s')$$

- This converges to V^* , even for loopy, stochastic MDP!
- Unlike policy iteration, there is no explicit policy.
- Intermediate value functions may not correspond to any policy.

Value Iteration



- $V_{k+1}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s')$
- Matrix Form: $V_{k+1} = \max_{a \in A} R^a + \gamma P^a V^k$

Example: Shortest Path

g			

Problem

Summary of Synchronous DP Algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- The algorithms are based on state-value function $V^\pi(s)$ or $V^*(s)$
 - The complexity is $O(mn^2)$ per iteration, for m actions and n states.
- Could also apply the same idea to Q functions $Q_\pi(s, a)$ or $Q^*(s, a)$
 - Complexity is $O(m^2n^2)$

Asynchronous Dynamic Programming

- Up to now, we have seen synchronous methods that update all states in parallel.
- *Asynchronous DP* backs up states individually, in any order
 - For each selected state, apply the appropriate backup
 - Can significantly reduce computation
 - *Guaranteed* to converge if all states continue to be visited.

Asynchronous Dynamic Programming

- Three simple ideas for asynchronous dynamic programming:
 - *In-place* dynamic programming
 - *Prioritized Sweeping*
 - *Real-time* dynamic programming

In-Place Dynamic Programming

- Synchronous value iteration stores two copies of value function

$$\forall s \in S, V_{new}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_{old}(s')$$
$$V_{old} \leftarrow V_{new}$$

- In-place value iteration only stores one copy of value function

$$\forall s \in S, V(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V(s')$$

Prioritized Sweeping

- *Intuition*: Backup the state with the largest remaining Bellman errors

- Formally, use magnitude of Bellman error to guide state selection:

$$\left| \max_a \left(r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V(s') \right) - V(s) \right|$$

- After each backup, update the Bellman error of affected states
 - However, to compute which states are affected, one requires knowledge of the reverse dynamics (predecessor states)
 - Can be implemented efficiently by maintaining a priority queue

Real-time Dynamic Programming

- *Idea:*

- update only states that are relevant to the agent.

- *Implementation:*

- Use the agent's experience to guide the selection of state.
- Formally, after each time-step s_t, a_t, r_{t+1}
- backup *only* the state s_t

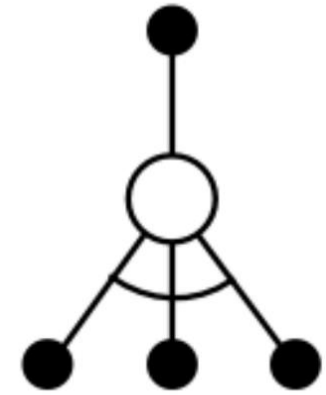
$$V(s_t) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{s_t s'}^a V(s')$$

Full-Width Backups

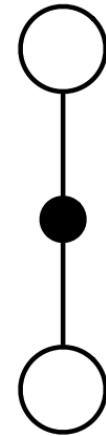
- DP uses *full-width* backups:
 - For each backup (sync or async)
 - Every successor state and action is considered
 - We can do this because we assume full knowledge of the MDP
- DP is effective for medium-sized tabular problems (million of states)
 - For larger problems, even if we know the MDP, every backup can be too expensive.
 - This is known as Bellman's *curse of dimensionality*
 - Note: number of states $n = |S|$ grows exponentially with the number of state variables.

Sample Backups

- In future lectures, we will consider *sample backups*.
 - Updates use sample rewards and sample transitions $\langle S, A, R, S' \rangle$ collected by a policy
 - Instead of having full knowledge of the reward function and transition dynamics.
- *Advantages:*
 - Model-free: no advance knowledge of MDP required
 - “Breaks” the curse of dimensionality through sampling
 - Cost of backup is constant, independent of $n = |S|$

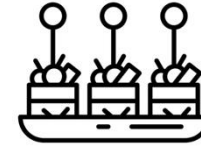


Now



Future Classes

Appetizer: Approximate DP



- Approximate the value function with a *function approximator* $\hat{V}(s; \mathbf{w})$, e.g. a neural network with parameters $\mathbf{w}$.
- *Idea:*
 - Pretend that $\hat{V}(s; \mathbf{w})$ is a good estimate of $V(s)$ and use it for DP.
- *Example algorithm:* Fitted Value Iteration
 - Sample a few states $\tilde{S} \subseteq S$ (for example using a policy)
 - For each state $s \in \tilde{S}$, estimate target value using Bellman Optimality Equation
$$\tilde{V}_k(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a \hat{V}(s'; \mathbf{w}_k)$$
 - Train the next value function $\hat{V}(s; \mathbf{w}_{k+1})$ using targets $\{\langle s, \tilde{V}_k(s) \rangle\}$

Technical Questions

- How do we know that value iteration converges to V_* ?
- Or that iterative policy evaluation converges to V_π ?
 - And therefore, that policy iteration converges to V_* ?
- Is the solution unique?
- How fast do these algorithms converge (if they do)?
- These questions are resolved by the *contraction mapping theorem*

Value Function Space

- Consider the vector space $\mathcal{V}$ over value functions
 - There are $|S|$ dimensions
 - $\forall s \in S, V(s)$ can have any value in $[-\infty, +\infty]$
 - Each point in this space fully specifies a value function $V(s)$
- We will show that the Bellman backup brings points in this space closer
 - For example, after each backup, the distance between $V_{old}(s)$ and $V_{new}(s)$ decreases.
 - Therefore, the backups *must* converge on a unique solution.

Bellman Expectation Backup is a Contraction

- See blackboard derivation

Contraction Mapping Theorem

Theorem (Contraction Mapping):

- For any metric space $\mathcal{V}$ that is complete (i.e., closed) under and operator $T(v)$, where T is a γ -contraction:
 - T converges to a unique fixed point
 - At a linear convergence rate of γ

Convergence Of Iter. Policy Evaluation and Policy Iteration

- The Bellman expectation operator T^π has a unique fixed point
- V_π is a fixed point of T^π (by Bellman expectation equation)
- By contraction mapping theorem:
 - Iterative policy evaluation converges on V_π
 - Policy Iteration converges on V_*

Bellman Optimality Backup is a Contraction

- Define the Bellman optimality backup operator T^* ,

$$T^*(v) = \max_{a \in A} R^a + \gamma P^a v$$

- This operator is a γ -contraction
 - It makes value functions closer by at least γ (similar to prior proof)

$$\|T^*(u) - T^*(v)\|_\infty \leq \gamma \|u - v\|_\infty$$

Converge of Value Iteration

- The Bellman optimality operator T^* has a unique fixed point
- V_* is a fixed point of T^* (by Bellman optimality equation)
- By contraction mapping theorem, value iteration converges to V_*
- Summary: Almost all algorithms presented today converge to the optimal solution!
 - It won't happen again during this class ☹️